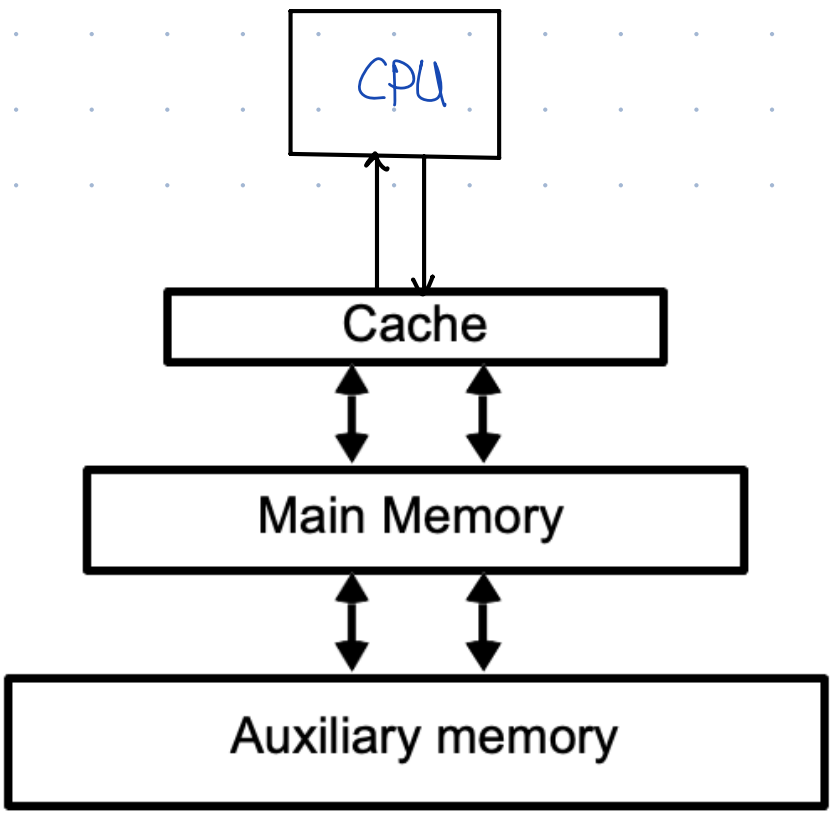
Memory is a major factor in determining performance.

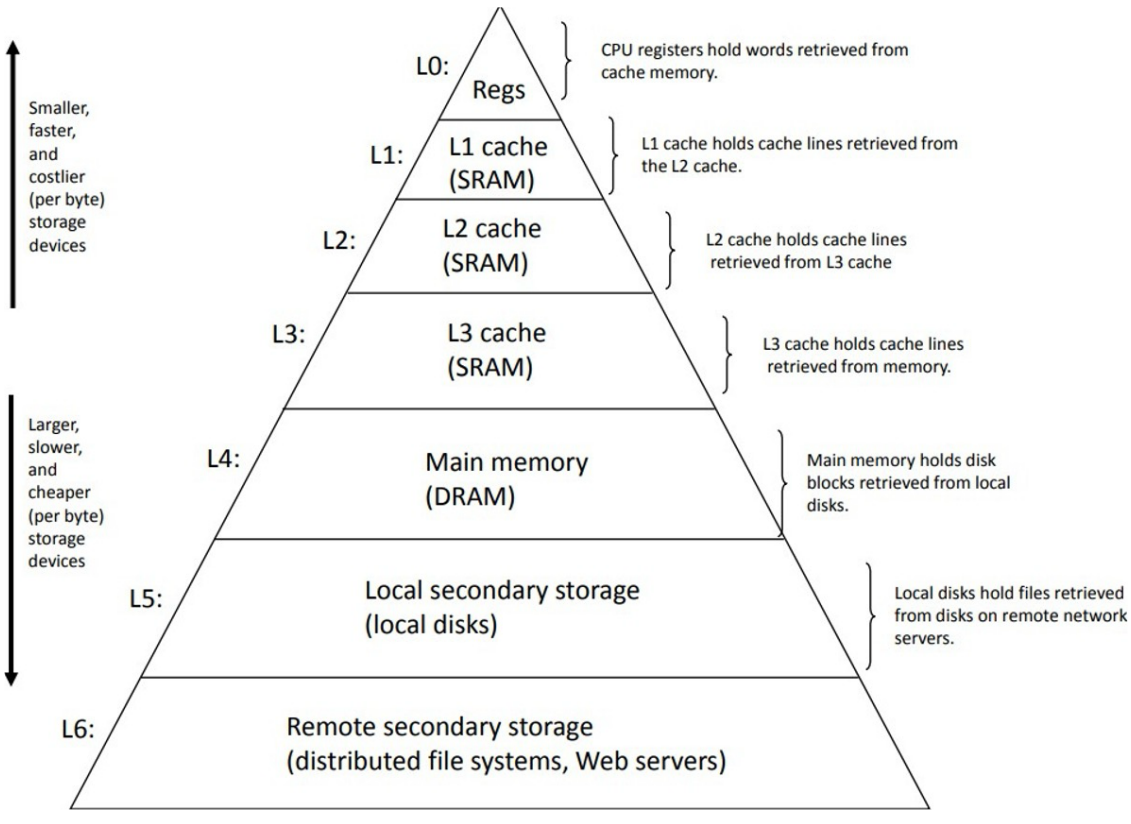
Principle of locality

↳ States that a processor accesses relatively small portions of programs at any instant time

Memory Hierachy:

↳ Consists of multiple levels of memory with different speeds & sizes.





↳ As the distance from the processor increases, so does size of memory

Why do we need memory hierachy?

1. To obtain the highest possible access speed while minimizing the total cost of the memory system.

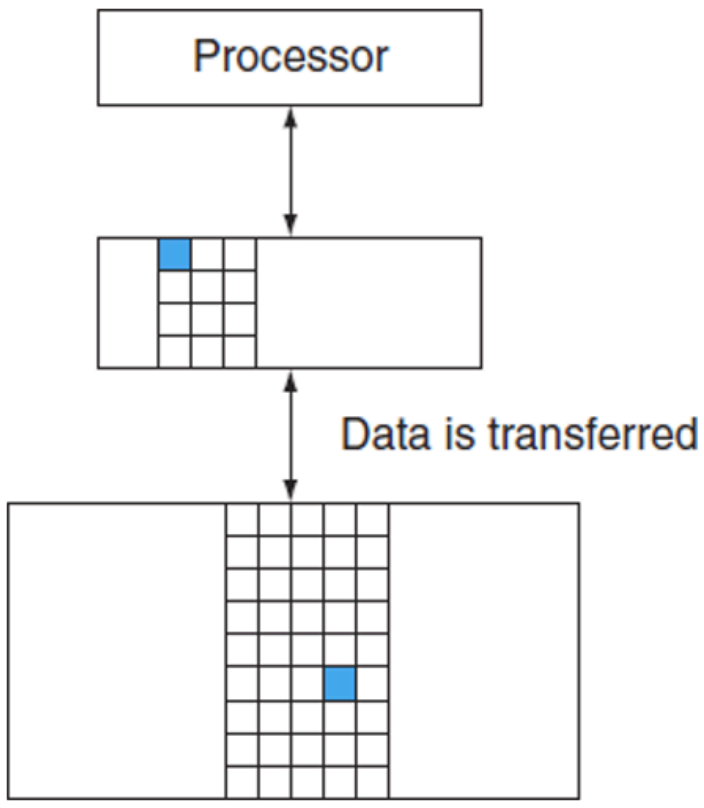
   ↳ Faster memories are more expensive per bit.

2. To take advantage of the "Principle of locality" by implementing the memory of a computer as a hierachy.

3. Performance

Data hierachy

- A level closer to the processor is a subset of of any level further away & all the data is stored at the lowest level.
- Data is copied only between two adjacent levels of memory at a time.



- Minimum unit of information present in a cache is called Block.
- If data requested for the processor appears in some block in the upper level, this is called hit.

If not, then the request is a miss.

Hit time:
The time to access the upper level of the memory hierachy, which includes the time needed to determine whether the access is hit or miss.

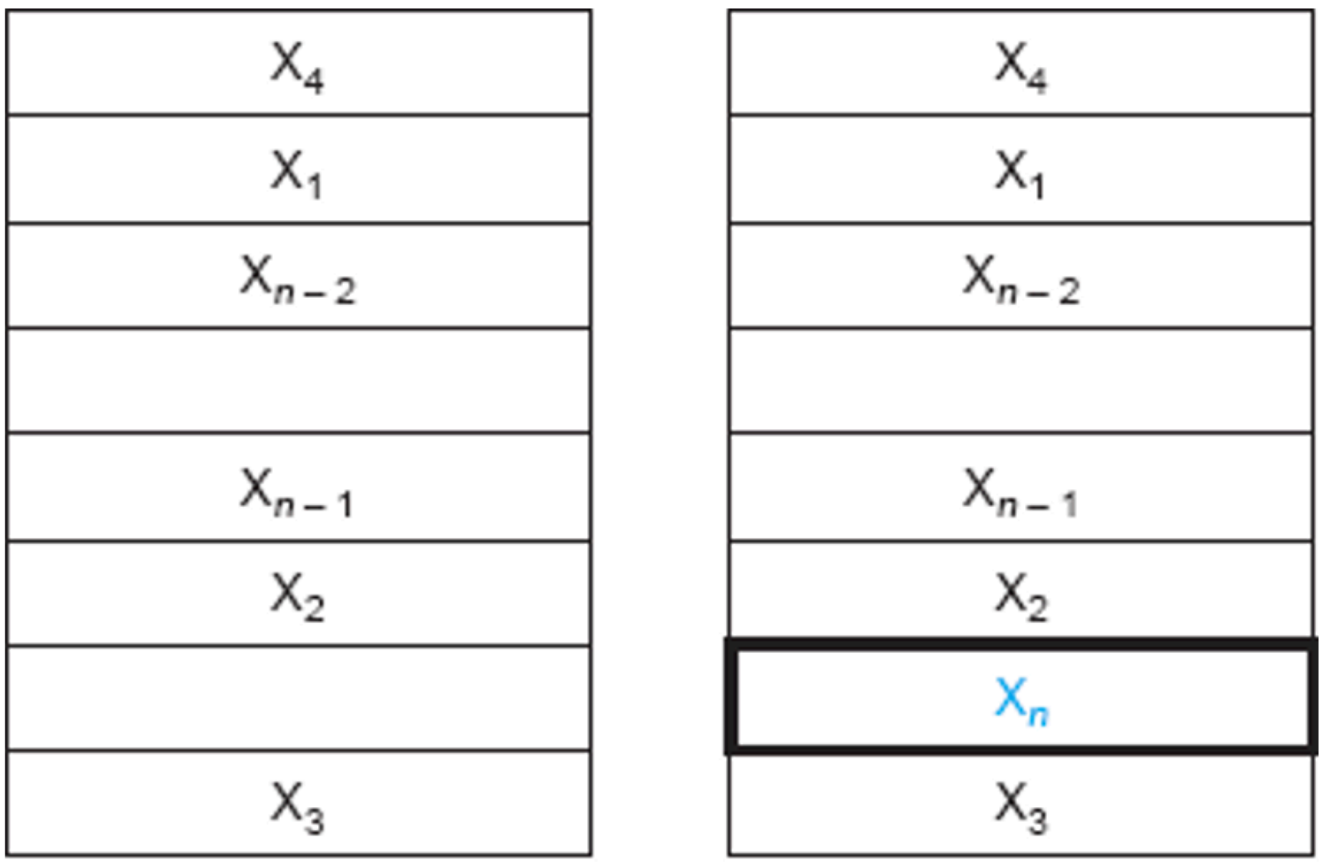
Miss penalty:
Time to replace a block in the upper level with wresponding block from the lower level plus the time to deliver this block to the processor.

Cache Basics

↳ Store main memory address + data at that address

Simple cache

Consider a cache in which the processor requests are each 1 one word & block is also one word.



a. Before the reference to X_n b. After the reference to X_n

NOTE:
why store block of data & not byte - in which processor requests for data?
This way we can take advantage of spatial locality

Direct mapped cache

↳ A cache structure in which each memory location is mapped to exactly one location in the cache, based on the address of the word in the main memory.

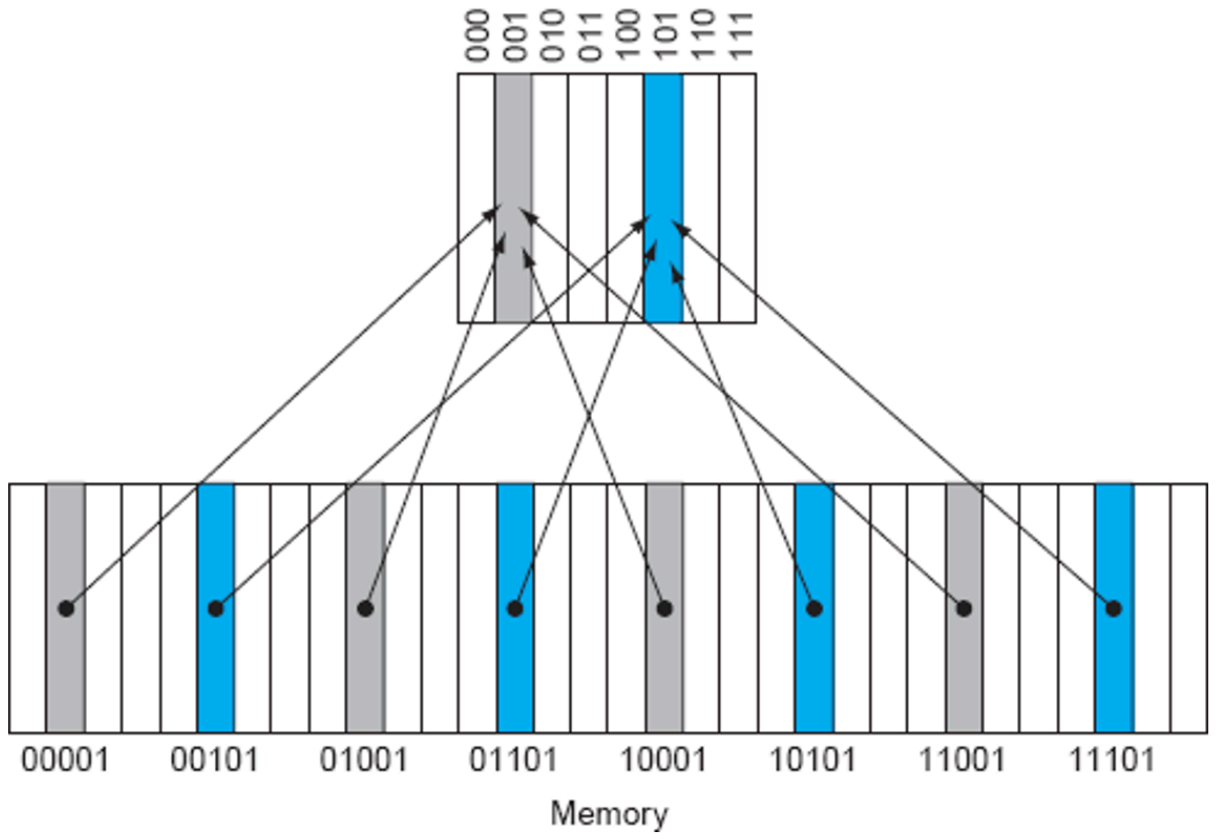
CACHE		
Tag	Index	Data

Memory address is divided into two parts: Tag & index
Why?

When processor accesses data from cache, we match index first & if index is a match we check tag or else we move on.

Lower bits of address : $\log_2(\text{size of cache}) = \text{index}$

Remaining upper bits = Tag



Why do we need tag?

Assume cache size = 8

So, index bits = $\log_3(8) = 3$

Now, 00001, 10001, 11001 all map to one index ie 001 thus we need tag to be sure if data is actually in cache.

Example,

Sequence of memory references		
Address of reference	Assigned cache block	Hit or miss
10110	10110 **mod** 8 = 110	miss
11010	11010 **mod** 8 = 010	miss
10110	10110 **mod** 8 = 110	hit
11010	11010 **mod** 8 = 010	hit
10000	10000 mod 8 = 000	miss
00011	00011 mod 8 = 011	miss
10000	10000 mod 8 = 000	hit
10010	10010 mod 8 = 010	**miss**
10000	10000 mod 8 = 000	hit
10011	10011 mod 8 = 011	miss

Cache		
Tag	Index	Data
10	000	M [10000]
	001	
10	010	M [11010]
~~00~~ 10	011	~~M[00011]~~ M[10011]
	100	
	101	
10	110	**M** [10110]
	111	

→ This is to illustrate

↳ In this case we replaced already existing data with this we got miss after matching tag.

Miss rate = $\frac{5}{9}$

= 55% Oops! So many!

Types of misses:

- Cold misses → No index matched
- Conflict misses: → index matched but not tag